ChatGPT

Computer Graphics – Ultimate Exam Prep

2D/3D Computer Graphics (Intro & Systems)

Must-Know Concepts

- **Definition:** Computer Graphics (CG) is the study of generating and manipulating images and animations via computers【4†L226-L234】【4†L251-L258】.
- **Main Tasks:** *Modeling* (describe geometry), *Rendering* (generate 2D image from 3D), *Animation* (change over time).
- **Applications:** Games, simulations, CAD/CAM, medical imaging, virtual training, user interfaces, etc.
- **Graphics Pipeline:** Comprises stages like modeling, transformation, lighting, rasterization, and display. (CG *pipeline* processes geometry → pixels.)
- **Graphics System Components:** Host CPU, GPU/graphics processor, large RAM, **frame buffer** (memory for pixel data), and display device (CRT/LCD).
- **Display Hardware:** Raster-scan displays update rows of pixels; resolution and refresh rate matter. (E.g. **frame buffer** stores pixel RGB values.)
- **Coordinate Systems:** Objects defined in world coordinates; transformed to *device* coordinates (via viewport mapping) for display.

High-Yield Notes

- **Vector vs Raster:** Vector images are resolution-independent; raster images (bitmaps) depend on pixel grid. CG often *rasterizes* vectors into pixels.
- **Memory vs Quality:** More color depth/higher resolution = more memory. Exams may ask trade-offs (speed vs quality).
- **Common Trap:** "CG = real-time only" is false – CG also used offline (film/VFX). Be clear on intent (e.g. photo-realism vs illustration).
- **Precision:** Integer math vs float – some algorithms (like Bresenham's) use integer arithmetic for speed.

Reinforcement Questions

1. **Q:** Define *computer graphics* and list its three main tasks.
   A: CG is the creation/manipulation of visual images and animations via computers【4†L226-L234】【4†L251-L258】. Main tasks are *Modeling* (geometry), *Rendering* (image generation), and *Animation* (time-variation).
   Explanation: The definition and tasks are core (per slides and sources). Model geometry → Render to pixels → Animate changes over time.

2. **Q:** What are typical components of a graphics system, and what is the role of the frame buffer?
   A: A graphics system includes a CPU/GPU, memory, and a frame buffer connected to a display. The **frame buffer** is a block of RAM holding pixel color values for the screen.

Explanation: The frame buffer stores the image that gets sent to the display. High resolution or color depth means a larger frame buffer (more memory needed).

3. **Q:** Compare vector graphics and raster graphics.
   A: *Vector graphics* store geometry (points, lines) mathematically and scale without quality loss; *Raster graphics* store pixels in a grid. Vector-to-display requires rasterization, while rasters have fixed resolution.
   Explanation: This tests understanding of how images are represented and why rasterization is needed.

4. **Q:** Why might one use lower color depth or resolution?
   A: To reduce memory usage and increase processing speed at the cost of image quality. It trades off detail (fewer bits or pixels) for performance or storage.
   Explanation: This links to "trade image quality for speed" (slide outcome). Common exam trap is forgetting the performance benefit of quantization.

5. **Q:** Name one example of a real-time CG application and one offline CG application.
   A: Real-time example: video game graphics. Offline example: CGI in movies or visual effects.
   Explanation: Show difference – games run on the fly, film rendering can take hours per frame.

Color Theory & Color Models

Must-Know Concepts

- **Color Model:** A mathematical scheme to represent colors as tuples (e.g. RGB triple). (*Color space* is specific values interpreted with viewing conditions.)【4†L226-L234】【4†L251-L258】
- **RGB (Additive):** Based on mixing Red, Green, Blue light. Used for displays. (0,0,0=black; 1,1,1 or 255,255,255=white.)
- **CMY(K) (Subtractive):** Cyan, Magenta, Yellow (and often Black for printers). Relation: C=1–R, M=1–G, Y=1–B. CMYK used in printing (K = black for efficiency). Undercolor removal in CMYK subtracts CMY components to add pure black ink, saving color ink.
- **HSV/HSL:** Hue (color type, angle 0–360°), Saturation (intensity of color), Value/Lightness (brightness). Useful for color selection.
- **YIQ/YUV:** Luma-chroma models (used in TV); Y = brightness, I/Q or U/V = color differences. (Often used for compression; not heavily tested unless slides covered them.)
- **RGB Cube:** Colors form a cube in RGB space. Diagonal from (0,0,0) to (1,1,1) goes from black to white with gray shades.
- **Gamma Correction (Trap):** Real displays do not respond linearly to voltage. In many problems, *assume* gamma = 1 (ignored) unless stated. (The slide warns "gamma correction usually ignored.")
- **Indexed Color & LUT:** An image can use an 8-bit index into a 256-color palette (LUT). To convert 24-bit (true color) to 8-bit, quantize each channel (e.g. 3 bits R, 3 G, 2 B) and compute palette index. E.g., index = (r_index*8* + *g_index)*4 + b_index.

High-Yield Notes

- **Gamma Caveat:** Always clarify if gamma correction is applied. Many solutions on exams assume no gamma (linear). The slide explicitly warns that gamma is usually ignored – an *exam trap*.

- **Color Model Distinctions:** Don't confuse HSV vs HSL; Hue is the same, but HSL's L (lightness) is different from HSV's V. Some instructors expect noting that HSL = 0 saturation still yields grayscale.
- **RGB vs CMY Relations:** A common mistake is mixing them up; remember CMY subtracts light, so white (all light) is (0,0,0) in RGB, but (0,0,0) in CMY yields white too (if defined as 1–RGB).
- **Undercolor Removal:** Adding K in printers avoids muddy colors. If asked, note CMYK allows using less ink.
- **Quantization:** When computing LUT index, remember to floor or use integer division. Errors often happen by off-by-one in slice calculation.

Reinforcement Questions

1. **Q:** What is the difference between additive and subtractive color models? Give an example of each.
   A: Additive (e.g. **RGB**) mixes light; adding R, G, B yields white light. Subtractive (e.g. **CMY**) mixes pigments or filters; mixing C, M, Y gives black (ideally).
   Explanation: RGB for screens, CMY for printing. Additive starts at black (no light), subtractive starts at white (paper).

2. **Q:** How does histogram equalization improve image contrast? Write the basic transformation formula.
   A: It remaps pixel intensities using the CDF of the image so that output intensities are spread across [0, L–1]. The transform is $T(i) = \mathrm{CDF}(i)$ (normalized)【19†L254-L262】【19†L273-L281】.
   Explanation: The CDF-based mapping makes output histogram uniform. Key formula: new intensity = cumulative probability up to old intensity.

3. **Q:** Convert 24-bit RGB (R=20, G=33, B=150) to an 8-bit indexed color using 3-3-2 bits per channel. Show steps.
   A: Divide 0–255 into 8 slices for R,G, 4 slices for B. R-index = ⌊20/32⌋ = 0; G-index = ⌊33/32⌋ = 1; B-index = ⌊150/64⌋ = 2. Then index = (0*8* + *1)*4 + 2 = 6.
   Explanation: Each 24-bit color channel is quantized. The formula is (R_slice * 8 + G_slice)*4 + B_slice for 3-3-2 bits.

4. **Q:** What is the RGB value of pure yellow? In CMY terms, what would that be?
   A: Yellow = (255,255,0) in RGB. In CMY, that's C=0, M=0, Y=1 (or 0% cyan, 0% magenta, 100% yellow) because Y = 1–B = 1.
   Explanation: Yellow light is red+green. In CMY, B=0 → Y=1; R=1→C=0, G=1→M=0.

5. **Q:** Give one advantage of the HSV model over RGB for user color selection.
   A: HSV separates color (hue) from intensity (value), so users can pick a hue and then adjust saturation/brightness. It's more intuitive (turn "color wheel" for hue).
   Explanation: In RGB, hue and brightness are entangled across three numbers, but HSV's H,S,V correspond more to human color concepts.

2D Geometric Transformations

Must-Know Concepts

- **Translation:** Moves points by (tx, ty):

$$x' = x + t_x, \quad y' = y + t_y.$$

- **Scaling:** Resizes about origin by (Sx, Sy):

$$x' = S_x\,x, \quad y' = S_y\,y.$$

- **Rotation:** Rotate by θ about origin:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}.$$

(From math sources: $R(\theta) = [[\cos\theta, -\sin\theta]; [\sin\theta, \cos\theta]]$【16†L12-L15】.)

- **Reflection:** Flip across an axis or line. Common reflections:

- Across x-axis: $x'=x$, $y'=-y$.
- Across y-axis: $x'=-x$, $y'=y$.
- Across line $y=x$: $x'=y$, $y'=x$.
- **Shearing:** Slants shape. For an *x-shear* by factor Shx: $x' = x + Sh_x\,y$, $y'=y$. For a *y-shear* by Shy: $x'=x$, $y' = y + Sh_y\,x$.
- **Homogeneous Coordinates:** Use (x,y,1) with 3×3 matrices to combine these transforms. Example: translation matrix = $\begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$.
- **Order Matters:** Transformations are non-commutative. E.g. rotating then translating ≠ translating then rotating (unless center is origin). Always apply matrix operations in correct sequence.

High-Yield Notes

- **Composition:** To rotate about point (a,b), translate (−a,−b), rotate, then translate back. Many exam traps involve rotating around origin vs arbitrary point.
- **Rotation Signs:** Positive θ typically means CCW (counterclockwise). If not specified, assume right-handed coordinate system (x right, y up).
- **Scaling Negatives:** Negative scaling factors cause reflections (flip axes). If asked reflection + scaling, note the sign effect.
- **Shear Distortion:** Shear changes shape but keeps area same. Horizontal shear moves x by fraction of y.
- **Matrix Multiplication Order:** When combining transforms in homogeneous form, *multiply the corresponding matrices in the reverse order of intended application*. (Students often get the order wrong.)

Reinforcement Questions

1. **Q:** Write the 2D rotation matrix for angle θ and use it to compute the image of point (1,0) when θ=90°.
   A: $R(90°) = \begin{pmatrix}0&-1\\1&0\end{pmatrix}$. Applying to (1,0) gives $(x',y')=(0,1)$.
   Explanation: Cos(90°)=0, sin(90°)=1. The matrix sends (1,0) to (0,1), a 90° CCW rotation.

2. **Q:** How would you reflect the point (3,5) across the line y=x?
   A: Swap coordinates: (x',y') = (5,3).
   Explanation: Reflection in y=x just swaps x and y. A trap is mixing up axes – memorize swap.

3. **Q:** Given point (2,3), apply shear with Shx=1 (x-shear), then translation (tx=1, ty=–2). What is the final point?
   A: First shear: $x' = 2 + 1*3 = 5$, $y' = 3$. After translation: $(x'',y'') = (5+1,\,3-2) = (6,1)$.*
   *Explanation:** Order is shear then translate. Check each step carefully.

4. **Q:** What happens if you apply scaling by (–1,1)?
   A: x' = –1·x, y' = 1·y. This reflects the object across the y-axis (and scales by 1).
   Explanation: Negative scale flips direction. A scale of (–1,1) is equivalent to reflecting in y-axis.

5. **Q:** Why do we use homogeneous coordinates for 2D transforms? Give one advantage.
   A: It allows representing translation as matrix multiplication (by adding a 3rd coordinate 1). This unifies all transforms (translation, rotation, scale, etc.) as matrix multiplications.
   Explanation: Without homogeneous coords, translation isn't linear. Homogeneous makes combining transforms straightforward (just multiply 3×3 matrices).

Rasterization (Scan Conversion)

Points and Lines

Must-Know Concepts

- **Pixels vs Coordinates:** Logical coordinates (real numbers) map to pixel grid. To *draw a point* at (x,y), usually round or floor to nearest integer pixel (floor gives largest integer $\leq$ coordinate).
- **DDA Line Algorithm:** Incremental method. Given endpoints P1(x1,y1), P2(x2,y2), compute steps = max(Δx,Δy). Increment x and y by small amounts each step. Formula uses floating arithmetic.
- **Bresenham's Line Algorithm:** Integer-based. Uses a **decision parameter** to choose whether to increment y (for gentle slopes). More efficient than DDA (uses only addition, no floating). Common pseudocode initializes error = 2Δy–Δx, etc.
- **Slope Cases:** For slopes >1 or negative slopes, reflect or swap axes. In practice, ensure walking in the dominant direction (steep vs shallow) to keep one axis as independent.
- **Endpoint Handling:** Typically plot both endpoints. Many implementations ensure P1 and P2 are both drawn.
- **Aliasing:** Raster lines show stair-step "jaggies". Antialiasing reduces this but usually not covered in basic CG.

High-Yield Notes

- **Comparisons:** DDA easier conceptually but slower (float mults). Bresenham is faster (only integer add/sub) – exam may ask which is more efficient.
- **Common Pitfall:** Forgetting to initialize error correctly or increment y at right times. Many students get off-by-one errors.
- **Continuity:** Must handle all octants: usually take absolute values of slope and step accordingly.

- **Traps:** If asked about line at 45°, both methods will work, but Bresenham's decision parameter equals 0 at some point, watch pixel choice.
- **Endpoint Order:** If x1 > x2, swap endpoints for algorithms expecting left-to-right scanning.

Reinforcement Questions

1. **Q:** Compare DDA and Bresenham line algorithms in terms of arithmetic operations.
   A: DDA uses floating-point increments (multiplication/addition each step). Bresenham uses only integer addition/subtraction (decision variable), making it faster and more efficient.
   Explanation: Bresenham's algorithm avoids costly floats. On older hardware, Bresenham was preferred for speed.

2. **Q:** Using Bresenham's algorithm, what decision would you make at the midpoint for a line from (0,0) to (5,3)? (Outline steps.)
   A: Δx=5, Δy=3, initial p = 2Δy–Δx = 2*3–5 = 1. Steps: at x=0->1 (p=1>=0) → y=1, new p = p + 2(Δy–Δx) = 1+2*(3–5) = -3. Next, p<0 → (x2,y1), update p = -3+2*Δy=3. Continue similarly. The sequence of chosen pixels approximates the line.
   *Explanation:** Illustrates decision variable updates. For brevity, the key is showing update logic.

3. **Q:** Why does Bresenham's algorithm produce a straight line without gaps?
   A: It always steps one pixel in x (and sometimes in y), ensuring continuous adjacency. The decision param ensures the best approximation to the true line each step.
   Explanation: Unlike simply rounding, Bresenham's logic keeps one pixel gap-free.

4. **Q:** How is a single point drawn in raster coordinates?
   A: Take the real (x,y) coordinates and use a rounding or floor method to choose the nearest pixel (e.g., Pixel(floor(x), floor(y))).
   Explanation: In practice we map float to int. The slide suggests Floor method (largest integer ≤ coord).

5. **Q:** For a line with negative slope, how do line-drawing algorithms handle it?
   A: Reflect or swap endpoints so slope is positive, draw the line, then reflect back. For Bresenham, one can increment y downward when needed (treat Δy as absolute and adjust sign).
   Explanation: Algorithms are usually described for slope 0–1; handling other octants requires symmetry or sign adjustment.

Circles (and Ellipses)

Must-Know Concepts

- **Circle Definition:** $x^2 + y^2 = r^2$. Only need compute 1/8th of circle (from 0° to 45°) and mirror to other octants.
- **Midpoint Circle Algorithm:** Similar to Bresenham line. Starts at (0,r). Uses decision variable $p = 1-r$; iterate x from 0 to y, update p: if p<0, choose E pixel (x+1,y); else choose SE pixel (x+1,y-1) and update p with different increments. Plot symmetry points.
- **8-way Symmetry:** From computed (x,y), plot points (±x,±y) and (±y,±x). (Total 8 per octant).

- **Ellipse Algorithm (if any):** Midpoint ellipse analog splits into two regions (slope 1 boundary). (Often not tested unless lecture explicitly taught.)
- **Parameters:** Only need integer radius. If center ≠ origin, translate after computation.

High-Yield Notes

- **First Octant:** Compute until x=y (45°) or decision changes. After that, swap roles of x and y.
- **Decisions:** The initial p = 1–r, update rules p_new = p + 2x + 3 (when E) or p + 2(x–y) + 5 (when SE). Off-by-one in these formulas is a common exam pitfall.
- **Symmetry:** Always mirror the points. Often students forget to plot all 8 symmetrical points.
- **Comparison:** Like lines, uses only adds (no multiplications) and integer operations – it's efficient.
- **Trap:** Remember to plot at least one point per octant; the trivial (r,0) and (0,r) points.

Reinforcement Questions

1. **Q:** Why do we only calculate 1/8th of a circle when using midpoint algorithms?
   A: Because of 8-way symmetry. Computing one octant and reflecting yields the full circle, greatly reducing work (by factor 8).
   Explanation: For each (x,y) in one octant, the other points (±x,±y, ±y,±x) are placed.

2. **Q:** What is the initial decision parameter p for the midpoint circle algorithm of radius r?
   A: $p_0 = 1 - r$.
   Explanation: Standard derivation (see textbooks). The first step uses this to choose pixel at (1,r) or (1,r–1).

3. **Q:** If at a given step $p<0$, which pixel do you choose and how do you update p?
   A: If $p<0$, the next pixel is East: $(x+1, y)$ and update $p = p + 2x + 3$.
   Explanation: This maintains p = difference from true circle. The formula comes from evaluating $p + 2x + 3$.

4. **Q:** How is ellipsis drawing different from circles? (Conceptual)
   A: An ellipse has two radii (rx, ry) and lower symmetry (4-way). The midpoint ellipse algorithm handles two regions (dx>dy vs dx<dy) separately. It uses different decision criteria because $x^2/a^2 + y^2/b^2 = 1$ is not rotationally invariant like a circle.
   Explanation: Unless needed, mention main difference: shape formula and 4-way symmetry.

5. **Q:** What is a common advantage of midpoint-based drawing algorithms?
   A: They use only integer arithmetic (efficient) and ensure continuity. Also, by using symmetry they reduce computations.
   Explanation: Contrast with naive approaches (like computing y via sqrt) which would be slow.

Region Filling

Must-Know Concepts

- **Boundary Fill:** Start from an interior seed point and expand until a boundary color is reached. Typically recursive: at pixel (x,y), if its color ≠ boundary and ≠ fill color, set to fill color and recurse on neighbors.
- **Flood Fill:** Similar but stops on a specified original color instead of a boundary color. (Flood fill fills all connected pixels of an original color.)
- **Connectivity:** 4-connected (up, down, left, right) or 8-connected (diagonals included). E.g. 4-connected has 4 neighbors, 8-connected has 8 neighbors.
- **Use Cases:** Fill closed shapes (like coloring in a polygon).

High-Yield Notes

- **Recursion Pitfall:** Naïve boundary/flood fill can cause stack overflow if region is large. (Non-recursive methods or scan-line fills avoid this.)
- **Seed Selection:** Must start inside region; if outside boundary color polygon, nothing happens.
- **Boundary vs Flood:** Boundary-fill requires a clear boundary color; flood-fill requires a known original color.
- **Exam Trap:** Don't confuse with clipping. Fill colors inside region only. Check connectivity definition in question (4 vs 8 connectivity changes result).
- **Implementation:** Ensure base case checks both boundary and already filled colors to avoid infinite loops.

Reinforcement Questions

1. **Q:** What's the difference between boundary fill and flood fill algorithms?
   A: Boundary-fill stops at boundary color; it fills until it reaches a boundary color. Flood-fill stops when it hits the original color it's replacing. In other words, boundary-fill uses a boundary value, flood-fill uses the initial region color as the stopping condition.
   Explanation: If filling a region until edge, boundary-fill is used. If filling by color match, flood-fill.

2. **Q:** Why might a flood-fill algorithm fill an unintended area?
   A: If the starting seed color is shared with other regions (not isolated), flood-fill will spread through all connected same-color areas. If boundary not properly enclosed, fill leaks.
   Explanation: Make sure boundaries or segmentation isolate areas, or choose correct seed.

3. **Q:** How can using 8-connected neighbors instead of 4-connected affect fill?
   A: 8-connected includes diagonals, so more pixels get filled (diagonal adjacency allows "leakage"). Using 4-connected may leave diagonal holes in a diagonally connected region.
   Explanation: 8-connectivity yields a "solid" fill; 4-connectivity can leave corner gaps.

4. **Q:** Outline a pseudocode snippet for boundary-fill with 4 neighbors.
   A:

```
Fill(x, y, boundaryColor, fillColor):
  if (color(x,y) ≠ boundaryColor AND color(x,y) ≠ fillColor):
    setPixel(x,y, fillColor)
    Fill(x+1, y, boundaryColor, fillColor)
    Fill(x-1, y, boundaryColor, fillColor)
    Fill(x, y+1, boundaryColor, fillColor)
    Fill(x, y-1, boundaryColor, fillColor)
```

Explanation: Recursively fills up/down/left/right until boundaryColor reached.

1. **Q:** What is a drawback of recursive fill methods, and how to mitigate?
   A: They can cause stack overflow on large regions. A mitigation is to use an iterative approach (e.g., scan-line fill, or breadth-first queue) instead of recursion.
   Explanation: Iterative methods manage memory better and can fill large areas without recursion depth issues.

Image Enhancement (Spatial Domain)

Contrast Stretching and Histogram Equalization

Must-Know Concepts

- **Histogram of an Image:** Graph of frequency of each intensity level.
- **Contrast Stretch (Linear):** Scale pixel values so the darkest becomes 0 and brightest becomes 255. Formula:

$$\text{new} = \frac{\text{old} - I_{\min}}{I_{\max} - I_{\min}} \times (L-1).$$

- **Histogram Equalization:** Spread intensities to flatten histogram. Compute the cumulative distribution function (CDF) of pixel values, then map each pixel to its CDF value scaled. Result: more uniform histogram. Key idea: $T(i) = \mathrm{CDF}(i)$ (normalized)【19†L254-L262】【19†L273-L281】.
- **Gamma Correction:** Adjust brightness nonlinearly: new = old^γ (or inverse). Useful for correcting display gamma or brightening/darkening images. (E.g., γ<1 brightens dark images.)

High-Yield Notes

- **Darkest/Brightest Pixels:** Contrast stretch requires finding I_min, I_max (often by scanning histogram endpoints). If a few pixels are extreme (outliers), stretching can oversaturate.
- **Histogram Equalization Trap:** It *maximizes* contrast globally but can produce weird effects (e.g., amplify noise). It's not local adaptive enhancement.
- **Color Images:** Usually apply equalization to intensity (grayscale or luminance), not per RGB channel, to avoid color shifts (unless instructed otherwise).

- **Gamma Usage:** If device has gamma ≠1, sometimes first linearize by gamma-1, do processing, then re-apply gamma. Exams might assume gamma=1 (no correction needed).
- **Sampled Data CDF:** Use normalized CDF (range 0–1) then scale to [0,L–1]. Rounding at end.

Reinforcement Questions

1. **Q:** A 3-bit grayscale image has intensity values {0,1,2,...,7}. If a pixel value 2 occurs 5 times, 4 occurs 10 times, and 6 occurs 5 times (total 20 pixels), what is the equalized value of intensity 4?
   A: CDF up to 4: P(0-4) = (5+10)/(20) = 15/20=0.75. New value = round(0.75*(7)) = round(5.25) = 5. Explanation:* Equalization maps 4 to 5. Shows using CDF formula.

2. **Q:** What effect does a 3x3 mean filter have on an image?
   A: It replaces each pixel with the average of its 3×3 neighborhood, smoothing (blurring) the image and reducing random noise, but it also blurs edges.
   Explanation: Mean filter is linear low-pass. It reduces variance (noise) but also reduces detail.

3. **Q:** Why is a median filter preferred for salt-and-pepper noise?
   A: Because median preserves edges better and effectively removes isolated extreme pixels. Replacing center with median kills outliers without blurring edges as much as averaging.
   Explanation: Median is non-linear and robust to impulse noise.

4. **Q:** Explain unsharp masking in image sharpening.
   A: Compute a blurred version of the image, subtract from original to get high-frequency (edges), then add back to original. Formula: I_new = I_orig + k*(I_orig – I_blur). Equivalently, use a Laplacian filter. Explanation:* It's called "unsharp" because you subtract a smoothed (unsharp) image to emphasize edges.

5. **Q:** What does applying a high-pass filter to an image do? Give an example of a high-pass mask.
   A: It enhances edges by removing low-frequency content. Example Laplacian mask: [[0,-1,0],[-1,4,-1],[0,-1,0]] highlights regions where pixel differs from neighbors.
   Explanation: High-pass allows fine details, edges; reduces smooth areas. Laplacian is common.

Image Processing (Frequency Domain)

Must-Know Concepts

- **Frequency Domain:** Represent image with Fourier transform. Low frequencies = smooth variations; high frequencies = edges/detail.
- **Discrete Fourier Transform (DFT):** Converts 2D image to frequency components. (No need formula on exam, just concept.)
- **Filtering:** In freq domain, *low-pass* filtering removes high frequencies (blur/smooth), *high-pass* filters remove low frequencies (sharpen/enhance edges).
- **Convolution Theorem:** Spatial convolution = multiplication in freq domain. Often faster for large kernels with FFT.
- **Filter Examples:** Ideal low-pass (circular cutoff), Gaussian low-pass, Butterworth, etc. Ideal often causes ringing (Gibbs phenomenon).

- **Spatial vs Frequency:** Some tasks (e.g., blurring) can be done in either domain. Multi-step: transform, multiply by mask, inverse transform.

High-Yield Notes

- **Space vs Frequency Complexity:** For large kernels, FFT can be more efficient (O(N log N) vs O(Nk^2)). The midterm has a question hinting "FFT more efficient for large kernels" (likely high confidence concept).
- **Edge Effects:** Frequency domain filtering assumes periodic extension; check image padding/truncation issues.
- **Traps:** Don't confuse high-pass with "removes edges" – actually it emphasizes edges. (See exam misconceptions in Lecture 8 Q's: "High-pass filtering removes edges" is false – it removes low frequencies, preserving edges.)
- **Sharpening in Frequency:** Adding a high-pass result to original image (like unsharp mask) yields enhancement.

Reinforcement Questions

1. **Q:** Describe what a 2D Gaussian low-pass filter does to an image and why it's separable.
   A: It smooths (blurs) the image by averaging pixel values with a Gaussian-weighted kernel, preserving edges less than an ideal filter. Separable because a 2D Gaussian = row-wise 1D Gaussian followed by column-wise 1D Gaussian.
   Explanation: Separable means convolution can be done in two passes (rows then columns), speeding computation.

2. **Q:** A filter has frequency response that is 1 for |f|<f_c and 0 otherwise. What is this filter and what artifact can it introduce?
   A: Ideal low-pass filter (circular cutoff). It can introduce ringing artifacts (Gibbs effect) near sharp edges in spatial domain.
   Explanation: The abrupt cutoff in freq causes oscillations in space.

3. **Q:** Why might one use the frequency domain to apply a large filter instead of doing it directly in the spatial domain?
   A: Because convolution with large kernels is faster via FFTs (O(N log N) operations) than direct convolution (O(N k^2)). Large 2D kernels are impractical to implement by direct summation.
   Explanation: Efficiency trade-off: overhead of FFT but gains for large sizes.

4. **Q:** In the context of image smoothing, what is the difference between a box filter and a Gaussian filter?
   A: A box (mean) filter assigns equal weight to neighbors; a Gaussian gives more weight to nearer pixels. Gaussian causes less blockiness and better preserves central intensity.
   Explanation: Gaussian is smoother in frequency (no sharp cutoff).

5. **Q:** What is the effect of multiplying an image by −1 before displaying it (in frequency domain)?
   A: Multiplying by (−1)^(x+y) (sometimes done to center freq) shifts the zero-frequency to center. This is standard practice but exam likely won't ask this detail.
   Explanation: Not crucial, skip if not needed.

Augmented & Virtual/Mixed Reality

Must-Know Concepts

- **Augmented Reality (AR):** Overlays computer-generated content onto the real environment 【11†L279-L287】. It *enhances* reality by adding virtual objects in real-time. (e.g. Pokemon Go on a camera view.)
- **VR vs AR:** VR is a *fully* virtual environment, replacing reality【13†L242-L250】. AR uses the real world as base. (AR adds to real; VR immerses in all virtual.)【11†L279-L287】
- **AR Types:**
- *Marker-based:* Recognizes a printed marker (e.g. ARToolkit) to place 3D model.
- *Markerless/Location-based:* Uses GPS/SLAM (Simultaneous Localization and Mapping) to align 3D content with real world (no pre-defined markers).
- *Projection-based:* Projects images onto real surfaces.
- **VR Levels:**
- *Non-immersive:* Desktop 3D simulation (not full immersion, e.g. PC driving sim).
- *Semi-immersive:* Big screens or partial VR (like flight simulators with a cockpit).
- *Fully immersive:* HMDs (head-mounted displays) or VR caves.
- **Mixed Reality (MR/XR):** A continuum blending real and virtual. MR usually means digital objects anchored to real world and able to interact with it. XR (Extended Reality) is an umbrella for AR+VR+MR.
- **360° Video:** Full spherical video capturing all angles. When viewed on HMD, user can look around. It's not interactive (camera motion only), but provides an immersive scene.

High-Yield Notes

- **AR Workflow:** AR uses tracking (often SLAM) to determine device pose. SLAM builds a map and tracks camera simultaneously. It's often asked in AR context.
- **Head Tracking (VR):** VR uses sensors (accelerometers, gyros) to update viewpoint continuously. Lag can cause nausea.
- **Applications:** AR in medicine (overlaying anatomy on patient), maintenance (instructions overlay). VR in training (flight simulators, surgery).
- **Traps:** Don't confuse 360° video with VR – it's passive (no interaction, just 360 panorama).
- **Field of View:** VR hardware often asks FOV or resolution specs. AR on smartphones usually smaller FOV than AR glasses.

Reinforcement Questions

1. **Q:** Define Augmented Reality and give one example of its use.
   A: AR overlays computer-generated images or info onto the real world in real-time【11†L279-L287】. Example: Smartphones showing virtual furniture in a room (e.g. IKEA Place app).
   Explanation: It's interactive and contextual. AR enables enhancing real scenes, unlike VR.

2. **Q:** What is SLAM and why is it important in AR?
   A: SLAM = Simultaneous Localization and Mapping. It builds a map of unknown environment while tracking device location. In AR, it allows markerless tracking (the device understands its position

relative to environment) without pre-set markers.
Explanation: Markerless AR and HoloLens use SLAM to place objects stably in real world.

3. **Q:** Describe one difference between Virtual Reality and Augmented Reality.
   A: VR immerses the user in a fully digital environment (blocks real world)【13†L242-L250】, while AR keeps the real world and overlays additional info【11†L279-L287】.
   Explanation: Key difference: AR augments reality, VR replaces it. This is often tested.

4. **Q:** What are the three levels of VR immersion?
   A: Non-immersive (desktop 3D, e.g. computer game), semi-immersive (CAVE or wide-screen simulators), fully immersive (HMD or dome with head tracking).
   Explanation: This taxonomy comes from slides (though exact terms may vary).

5. **Q:** What is a 360° video and how does it relate to VR?
   A: A 360° video is a spherical video that captures all directions. When viewed on VR goggles, you can look around in the fixed viewpoint (the camera can't move). It's a passive VR experience (no motion inside scene).
   Explanation: It's like watching a pre-recorded real scene from all angles.

Computer Vision Basics

Must-Know Concepts

- **Definition:** Computer Vision (CV) is about **interpreting** images and videos – enabling computers to "see" and extract information. Unlike CG (create images) and IP (enhance images), CV *analyzes* images.
- **Pipeline:** Steps often: *Acquire* image → *Preprocess* (noise reduction) → *Feature Extraction* (edges, corners, textures) → *Recognition/Classification/Detection* → *Decision* (class label, object identity, etc.).
- **CV Tasks:**
- *Classification:* Identify main object (one label for whole image). E.g. "This is a cat."
- *Object Detection:* Find and classify objects with bounding boxes. E.g. "Person at (x,y), Car at (u,v)."
- *Segmentation:* Label each pixel (e.g. background vs object; semantic segmentation or instance segmentation).
- *Edge Detection:* Find pixel-level edges (Sobel, Canny). Preliminary step for many tasks.
- *Thresholding:* Simple segmentation by intensity cutoff (binary image creation). Useful if objects clearly brighter/darker.
- **Edge Detection (Example):** Filters (like Sobel) compute gradients; Canny applies Gaussian blur + gradient + non-max suppression + hysteresis thresholding.
- **Segmentation:** Can be global (thresholding) or more advanced (region growing, clustering, deep learning).

High-Yield Notes

- **Data Difference:** CG creates synthetic data; CV deals with noisy real data (lighting, occlusions). Emphasize that.
- **Feature vs Template:** Early CV used hand-crafted features; modern CV (not likely covered) uses ML/AI. But exam likely sticks to basics (edges, thresholds).

- **Common Trap:** Don't mix "detection" with "segmentation." Detection boxes vs pixel classification.
- **Example Application:** Brain tumor detection via threshold (slides gave that example). They might ask to describe steps: e.g. "Threshold MRI to segment tumor vs background."
- **Multiple Objects:** Classification handles one object per image; detection can handle many; segmentation yields precise shape.

Reinforcement Questions

1. **Q:** What is the main goal of computer vision? How is it different from image processing?
   A: CV's goal is to extract meaningful information from images (interpret content). Image Processing mainly modifies images (improve quality, transform). CV *analyzes*, IP *enhances*.
   Explanation: Distinguish generation/enhancement vs analysis. Example: IP might denoise; CV might detect a tumor in the denoised image.

2. **Q:** A system labels an image as "cat" or "dog." Is this classification, detection, or segmentation?
   A: Classification (assign one label to the whole image).
   Explanation: There's no localization or per-pixel mask, just global label.

3. **Q:** How does object detection differ from semantic segmentation?
   A: Object detection finds objects and draws bounding boxes. Semantic segmentation labels every pixel with a class. Detection gives coarse location; segmentation gives exact shape.
   Explanation: Key difference: segmentation is pixel-level classification.

4. **Q:** Explain the role of thresholding in CV, with an example.
   A: Thresholding converts a grayscale image to binary by choosing a cutoff (T). E.g. to detect bright objects, set pixels > T = object (1), else background (0). Example: simple tumor detection on MRI.
   Explanation: Simple but effective if object is much brighter/darker.

5. **Q:** What is an edge detector (e.g. Sobel or Canny) used for?
   A: To find significant intensity changes (edges) in an image. Edges often correspond to object boundaries or features.
   Explanation: Preprocessing step, often feeding into higher-level tasks (like segmentation or recognition).

Previous Exams Analysis

Topic Frequency Analysis

We examined finals/midterms (2021–2025). The most frequently tested topics (present in **current syllabus**) are:

- **Augmented Reality (AR):** Appears *consistently high* (e.g. 14–15 mentions/year after 2021). AR is a top priority.
- **Graphics Systems & Coordinate Concepts:** (Fundamentals: frame buffer, pixels, transformations). Frequent mentions (3→9→9 per year, rising).

- **Color Models & Theory:** (RGB, CMY, HSV, color spaces). Rare in 2021-22 but jumped (0→12 mentions in 2023 onward). Now important.
- **2D Transformations:** (Translation/rotation/reflection, etc.) Constant: ~3–6 mentions yearly. Maintaining stable importance.
- **Virtual Reality (VR):** Some focus (a few questions); often paired with AR sections or definitions.
- **Computer Vision & Image Processing:** Rare (very few mentions); likely lower priority given syllabus. (We see 1–2 mentions of thresholding or segmentation, mostly context.)

Trends: AR/VR topics are very hot (rising and high frequency). Color theory usage increased (perhaps new to syllabus recently). Rasterization (scan conversion) has low frequency historically (few line-circle-fill questions in old exams), but they are core syllabus – they could appear, but past suggests moderate priority. Image processing appears infrequently in exams, but still worthy as Tier C.

Topic	Appearances (2021–2025)	Trend/Notes
Augmented Reality (AR)	~15 per year (2023–25)	**Increasing**
Virtual/Mixed Reality (VR/MR)	few mentions	**Stable low**
Graphics Systems & Displays	~3→9 per year	**Increasing**
Color Models/Theory	0→12 per year	**Increasing**
2D Transformations	~3–6 per year	**Stable**
Rasterization (Lines/Circles)	Very few	Decreasing
Region Filling	None	Not tested yet
Image Enhancement (filters)	Very few	Not tested often
Computer Vision Tasks	0–2 mentions	Very low

Pattern Analysis

- **AR/VR Questions:** Often definition or concept. E.g. *"Name AR types"*, *"SLAM vs markers"*, *"VR immersion levels"*. They *repeat vocabulary* and differences (AR vs VR). They test conceptual understanding, not numeric calculation.
- **Color/Display:** Newer trend: color spaces often grouped (RGB vs CMY vs HSV). Past Qs may ask for conversions or properties (e.g. "subtract CMY from 1"). Watch for "gamut problems" or "undercolor removal" type traps.
- **Transformations:** Common style: *"Given a sequence of transforms, find final matrix or coordinates."* Or conceptual *"properties of transform matrices"*. Professors often invert order or change pivot – check slides on center-of-rotation (traps).
- **Raster Lines:** Rare but when asked, usually *algorithmic*: e.g. *"What is decision parameter after k steps?"* or *"Compare DDA vs Bresenham"*. Sometimes fill-in-a-table of coordinates for a line.
- **Circle Algorithms:** Even rarer. Possibly *"midpoint circle vs parametric"* or *"point symmetry logic."* Might test understanding of octant plotting.
- **Histogram/Filters:** When used, in multiple-choice quizzes in slides. But few exam Qs. If asked, likely concept: "Which filter preserves edges best?" (Answer: median) or "What does histogram equalization do?".

- **Computer Vision/Thresholding:** Rare single questions, often conceptual *"CV vs CG vs IP?"* or *"Thresholding example."* The slides had a few MCQs (like "Thresholding outputs binary image").

Why they appear: AR/VR and color are newer syllabus areas with broad interest. Transforms and displays are fundamental CG topics. Repetition of AR/VR suggests professor's focus (Tier A). Rasterization, filling, image processing have been less emphasized historically (Tier B/C) but still core taught topics. Possibly filling will show up on exam once.

High-Priority Focus Areas

- **Tier A (Highest Priority):** Augmented Reality/Virtual Reality (AR/VR/MR/XR), Color Models, Graphics Systems/Display, 2D Transformations.
  Justification: These topics have the most frequent exam appearances and are heavily covered in lectures. AR/VR are consistently tested; professor emphasizes differences and types. Color theory and transformation fundamentals are also repeated and appear critical.

- **Tier B (Important):** Rasterization (lines, circles) and core image processing (filtering, histogram).
  Justification: While less frequent historically, these are core to the syllabus. Questions on line algorithms or smoothing/sharpening could appear. Should prepare well but prioritize after Tier A.

- **Tier C (Lower Priority):** Region filling algorithms, advanced vision (classification/detection details), 360° video.
  Justification: No past exam questions on filling or 360° specifically; likely less tested. Understanding is still needed, but after A/B. If time is limited, focus on A/B first.

Exam Prediction

Confidence Levels: Based on syllabus emphasis and past trends.

- **(High)** *AR/VR Concept Question:* e.g. "Describe how augmented reality differs from virtual reality and give one example of each."
  Reason: AR/VR repeatedly appear; definitions and differences are asked almost yearly. Lecture slides explicitly compare AR vs VR【11†L279-L287】【13†L242-L250】.

- **(High)** *2D Transform Sequence:* e.g. "Find the composite transformation matrix for a rotation of 30° about point (x0,y0) followed by a translation."
  Reason: 2D transform formulas and composition are core; many exam Qs involve combining transforms or specific cases (rotate about origin vs other point).

- **(Medium)** *Color Conversion:* e.g. "Convert RGB(255,128,0) to HSV." or "Given a color, compute its LUT index in 8-bit."
  Reason: Color is now heavily covered; slides included examples of LUT indexing. Likely an example calculation (similar to our reinforcement Q).

- **(Medium)** *Bresenham vs DDA:* e.g. "Explain Bresenham's algorithm for lines; why is it faster than DDA?"
  Reason: Lines seldom appear, but this concept was taught explicitly; could be a short-answer or MCQ.

- **(Medium/Low)** *Histogram Equalization:* e.g. “What does histogram equalization do to an image’s contrast?” Possibly accompanied by a small histogram example.
  Reason: One question on histogram was in slides; less tested in past, but fair chance for a conceptual question.

- **(Low)** *Region Filling Pseudocode:* e.g. “Give pseudocode for flood-fill with 4-neighbors.”
  Reason: Lecture included it, but no past exam Q. Could appear, but less certain.

- **(Low)** *360° Video:* e.g. “What is a 360° video and how is it used?”
  Reason: Lecture had it, but historical exam frequency is zero. Could be a single MCQ or a small part of a larger AR question.

For each prediction, support patterns: AR/VR predictions from frequent past questions (Professor loves XR topics). Transforms from fundamental importance. Color from new slides + patterns. Low-confidence if no direct precedent but aligned with recent syllabus.

Mock Exam

Instructions: Solve all parts. Use lecture concepts; show relevant formulas or reasoning.

1. **(Easy)** *Computer Graphics vs Computer Vision:* Briefly differentiate CG, Image Processing, and Computer Vision in terms of purpose and outputs.
2. **(Easy)** *RGB and CMY:* Given an RGB color (0.2, 0.5, 0.7), compute the corresponding CMY values.
3. **(Easy)** *Histogram Stretch:* An 8-bit image has min intensity 50, max 200. What linear transformation maps this range to [0,255]?
4. **(Medium)** *2D Transform:* Apply a 90° rotation about the point (2,3) to the point (5,7). Show steps (translate, rotate, translate back).
5. **(Medium)** *Line Rasterization:* Draw by calculation (DDA or Bresenham) the pixels for the line from (2,3) to (7,5). (List coordinates of chosen pixels.)
6. **(Medium)** *Augmented Reality:* List and briefly explain three types of AR.
7. **(Hard)** *Circle Drawing:* Outline Bresenham’s midpoint circle algorithm steps for radius r=5. Compute the first few points.
8. **(Hard)** *Image Filtering:* Given a 5×5 grayscale patch, apply a 3×3 mean filter and a 3×3 median filter to the center pixel. (Use given values.)
9. **(Hard)** *Virtual Reality:* Describe three levels of VR immersion and give an example device or scenario for each.
10. **(Hard)** *Composite Transform:* Find the single 3×3 homogeneous matrix that translates by (1,2), then rotates 45°, then scales by 2 in x and 0.5 in y (about the origin).

Answer Key

1. **Answer:** CG generates images; IP enhances/filters images; CV interprets/analyzes images.
   Explanation: CG = synthesis (e.g. rendering objects). IP = modify existing image (denoise). CV = extract info (detect faces).

2. **Answer:** CMY = (1–R, 1–G, 1–B) = (0.8, 0.5, 0.3).
   Explanation: Subtractive complement of RGB (assuming normalized [0,1]).

3. **Answer:** New = round((old–50)*(255/(200–50))). Formula: new = (old–50)*1.7 (approx).
   Explanation: It linearly maps 50→0 and 200→255. (Exact factor=255/150 = 1.7).

4. **Answer:** Translate (2,3) to origin: (x–2, y–3): (5–2,7–3)=(3,4). Rotate 90°: (x',y')=(-4,3). Translate back: add (2,3): (x'',y'') = (-4+2,3+3)=(-2,6).
   Explanation: Rotation matrix used is [0 -1;1 0]. Check each step.

5. **Answer:** Δx=5, Δy=2, Δx>Δy. Using Bresenham: start at (2,3). p0 = 2Δy–Δx = 4–5 = -1.

6. Step 1: (3,3), p<0 → p1 = p0+2Δy = -1+4=3.

7. Step 2: (4,4), p1>=0 → p2 = p1+2(Δy–Δx) = 3+2*(2–5) = -3.
8. Step 3: (5,4), p2<0 → p3 = -3+4 = 1.
9. Step 4: (6,5), p3>=0 → p4 = 1+2*(2–5) = -5.

10. Step 5: (7,5).
    Pixels: (2,3), (3,3), (4,4), (5,4), (6,5), (7,5).
    Explanation: This sequence approximates the line. Verify symmetrical stepping.

11. **Answer:** (a) *Marker-based AR:* Uses visual markers to place content (e.g. QR codes for AR). (b) *Markerless/SLAM:* Uses camera/IMU to map environment (e.g. AR navigation on phone). (c) *Projection AR:* Projects images onto surfaces (e.g. interactive tabletop display).
    Explanation: Each type augments differently; lecture listed these categories.

12. **Answer:** Radius=5. Start at (x=0,y=5), p=1–5=–4.

13. (0,5): plot. p<0 → next (1,5), p1=−4+2*0+3=−1.

14. p1<0 → next (2,5), p2=−1+2*1+3=4.
15. p2>=0 → next (3,4), p3=4+2*(2−5)+5=4+2*(−3)+5=−1.
16. p3<0 → (4,4), p4=−1+2*3+3=8.

17. Continue until x>y. Points in first octant: (0,5), (1,5), (2,5), (3,4), (4,4). (Then symmetric).
    Explanation: This follows midpoint logic. (Full fill would mirror these points to get the circle.)

18. **Answer:** Suppose center value= C, neighbors shown (example needed). For mean: replace center with average of 3×3 values. For median: sort 9 values, pick middle.
    Example: If patch =

```
10  20  30
20  50  40
30  40  60
```

Mean: (sum=300) avg=33 (rounded). Median: sorted [10,20,20,30,30,40,40,50,60], median=30.
Explanation: Mean blurs uniformly, median preserves a typical neighbor.

1. **Answer:** (a) *Non-immersive:* PC or console games (simple 3D on screen). (b) *Semi-immersive:* Flight simulators with partial cockpit or large screen. (c) *Fully immersive:* VR headset (e.g. Oculus Rift) gives 360° view and tracking.
   Explanation: These align with lecture: lower levels offer partial immersion, full requires HMD.

2. **Answer:** Translation(1,2): $T=\begin{pmatrix}1&0&1\0&1&2\0&0&1\end{pmatrix}$. Rotation45°: $R=\begin{pmatrix}c&-s&0\s&c&0\0&0&1\end{pmatrix}$ with $c=\cos45°=0.7071$, $s=0.7071$. Scaling: $S=\begin{pmatrix}2&0&0\0&0.5&0\0&0&1\end{pmatrix}$. Composite = $S \cdot R \cdot T$ (matrix multiply in that order).
   Numerically:

$$M = \begin{pmatrix}2 & 0 & 0\\ 0 & 0.5 & 0\\ 0 & 0 & 1\end{pmatrix} \begin{pmatrix}0.7071 & -0.7071 & 0\\ 0.7071 & 0.7071 & 0\\ 0 & 0 & 1\end{pmatrix} \begin{pmatrix}1 & 0 & 1\\ 0 & 1 & 2\\ 0 & 0 & 1\end{pmatrix}.$$

   (Final numeric multiplication yields the answer matrix.)
   Explanation: Multiply as 3×3. Order: last applied is leftmost or vice versa depending convention; careful to match lecture convention.
